SSR GLOBAL SKILL PARK
BHOPAL

PROJECT REPORT

CONTENT SAATHI

AI-Powered Daily Content Assistant for Small Businesses

Module 7 – Infocom Project

Prepared By
Kuldeep, Paras Mehra, Anil Kumar and Gaurav Yadav

Submitted To
SSR Global Skill Park, Bhopal

Academic Year: 2025-26

CONTENT SAATHI

PROJECT REPORT

AI-Powered Content Assistant for Small Businesses

Project information	Project details
Student name(s)	Kuldeep Sanodiya, Paras Mehra, Anil Kumar, Gaurav Yadav
Enrollment number(s)	SSRGSP/NET/2025/2/054, SSRGSP/NET/2025/2/071
Batch / course	Nov-4B / ICT
Institute	Sant Shiromani Ravidas Global Skills Park (SSRGSP)
Project guide	Mr. Siddharth Singh
Project duration	16-10-2026 and 23-10-2026
Academic year	2025-26

SUBMITTED TO: Mr. Siddharth Singh,

Advanced Networking and System Administration

Abstract / Executive Summary

Content Saathi is a Python-based AI application that helps small business owners create promotional content quickly and easily. Small businesses need regular promotion, but writing posts every day takes time, not everyone writes well, and hiring professional writers can be expensive. Content Saathi addresses this by using Google Gemini AI to generate content from the shop's details and the owner's instructions. The owner interacts with the system through a Telegram bot: they can request posts, ask questions and receive AI responses in the same chat.

Generated posts are stored in a SQLite database with a status of Draft, Approved or Rejected. The owner can review each draft, regenerate it if it is unsatisfactory, and approve or reject it, so the owner always stays in control of what is finally used. The system supports Telegram, Instagram and Google Business post formats, scheduled content generation at configured times, and festival-based content that uses verified festival dates. The backend is built with Python and FastAPI and served with Uvicorn; shop information and configuration are kept in JSON files and secrets are managed with python-dotenv.

Keywords: AI content generation, Google Gemini, Telegram bot, FastAPI, SQLite, small business, draft approval, scheduled generation.

Table of Contents

Abstract / Executive Summary

1. Introduction and Project Description
2. Requirements Specification
3. Project Plan, Schedule and Progress Report
4. AI Design and Architecture
5. Implementation
6. Testing, Test Cases and Results
7. Deployment
8. Problems Encountered and Solutions
9. Results, Limitations and Future Scope
10. Conclusion and Learning
11. References
12. Appendices

1. Introduction and Project Description

1.1 Project Overview

Content Saathi ("Saathi" meaning companion) is designed to be a simple everyday companion for shop owners who need to keep promoting their business. It combines a generative AI model (Google Gemini) with a familiar chat interface (Telegram) so that an owner without technical or writing skills can request, review and approve promotional posts from a phone. The project's main goal is to make content creation faster, easier and more accessible for small businesses.

1.2 Background and Problem Statement

Small businesses depend on regular promotion, such as festival greetings, offers and announcements, to stay visible to customers. However, producing content daily is difficult:

- Creating daily posts takes time away from running the business.
- Not every owner is comfortable writing marketing copy.
- Professional writers or agencies can be expensive for a small shop.
- Maintaining a regular promotion schedule is hard.
- Tracking which content was created, approved or discarded is difficult when done manually.

1.3 Proposed Solution

Content Saathi uses AI to generate promotional content from the shop's own details and the owner's request, and uses a simple Telegram-based system to manage that content. Every generated post is saved as a draft that the owner can regenerate, approve or reject. Posts can also be generated automatically at configured times, and festival posts can be created using verified festival dates.

1.4 Objectives

1. Generate promotional content using AI.
2. Allow content requests through Telegram.
3. Provide AI-powered responses to user queries.
4. Support Telegram, Instagram and Google Business post formats.
5. Allow content to be regenerated when needed.
6. Store generated posts in SQLite.
7. Provide draft approval and rejection.
8. Support scheduled content generation.

1.5 Project Scope

Included in the project scope	Outside the project scope
Generating promotional posts from shop details and user instructions	Guaranteeing sales, reach or customer growth
Telegram bot for requests, queries, regeneration and review	Publishing automatically to social media without the owner's approval
Formats for Telegram, Instagram and Google Business	Managing paid advertising campaigns
Storing posts and their status (Draft / Approved / Rejected) in SQLite	Collecting customer personal data
Scheduled and festival-based content generation	Multi-shop or multi-account management (listed as future scope)

1.6 Intended Users and Use Case

The intended users are owners and staff of small shops and local businesses who need regular promotional content but have limited time, writing skills or budget.

Example use case: A shop owner sends a request in Telegram for an Instagram post about a weekend offer. The system combines the shop's saved details with the request, asks Gemini to generate the post, saves it as a draft and sends it back. The owner asks for a new version, is satisfied with the second one, and approves it. The status in the database changes from Draft to Approved.

1.7 Success Criteria

Criterion	Measurement method	Target / actual
Posts reflect the shop details and request	Review a set of sample requests against the saved shop information	[Set target and record actual result]
Content is available in the supported formats	Generate Telegram, Instagram and Google Business posts and compare formats	[Record result]
Regeneration works	Request regeneration of a draft and check that a new version is returned	[Record result]
Status is stored correctly	Approve and reject posts and check the database record	[Record result]
Owner remains in control	Confirm that every generated post waits for approval or rejection	[Record result]

2. Requirements Specification

2.1 Requirements Gathering

Requirements were derived from the problems faced by small businesses in creating regular promotional content and from the project objectives set by the team. [State which methods were actually used, for example discussions with local shop owners, review of similar tools, or guidance from the project guide, together with the number of participants and the main findings.]

2.2 Functional Requirements

ID	Requirement	Priority	Status
FR-01	The system shall generate promotional content using Google Gemini AI from shop information and user instructions.	Must	[Actual status]
FR-02	The system shall accept content requests and commands through a Telegram bot.	Must	[Actual status]
FR-03	The system shall answer user queries using the configured Gemini model.	Should	[Actual status]
FR-04	The system shall support Telegram, Instagram and Google Business post formats.	Must	[Actual status]
FR-05	The system shall allow the user to regenerate a draft that is not satisfactory.	Must	[Actual status]
FR-06	The system shall store generated posts and their status in a SQLite database.	Must	[Actual status]
FR-07	The system shall allow the user to approve or reject a draft.	Must	[Actual status]
FR-08	The system shall generate content automatically at configured times.	Should	[Actual status]
FR-09	The system shall create festival-based posts using verified festival dates.	Should	[Actual status]

2.3 Non-Functional Requirements

Category	Requirement
Usability	A shop owner can use the system through simple Telegram messages and buttons without technical knowledge.
Reliability	If the AI service or Telegram is unavailable, the system reports the problem clearly instead of pretending the action succeeded.
Security and privacy	API keys and the bot token are stored in environment variables (python-dotenv), not in source code; only shop information needed for content generation is stored.
Maintainability	Shop information and configuration are kept in JSON files so they can be changed without altering the code.
Performance	Response time is measured using a stated number of test runs.
Responsible AI	Generated content is a draft; the owner reviews it before use, and the system does not guarantee business results.

2.4 Hardware and Software Requirements

Component	Details	Purpose
Hardware	Laptop / PC with [RAM / CPU specifications]	Development, testing and running the application
Programming language	Python [version]	Main application logic
AI model	Google Gemini AI [model name and version]	Content generation and AI responses
Backend framework	FastAPI	Backend API development
Server	Uvicorn	Running the FastAPI server
User interface	Telegram Bot API	Communication with the user
Database	SQLite (database/content_saathi.db)	Storing generated posts and their status
Configuration	JSON files; python-dotenv	Shop information, settings and environment variables
Scheduling	Python scheduler	Scheduled content-generation tasks
Network	Internet connection	Access to Telegram and the Gemini API

2.5 Constraints and Assumptions

- The system depends on the availability, rate limits and quotas of the Gemini API and the Telegram Bot API.
- AI output can vary between requests, so every post needs the owner's review.
- The shop details saved in the JSON configuration are assumed to be correct and up to date.
- Festival posts rely on verified festival dates; the dates are assumed to be accurate as entered or verified.
- An internet connection is required for AI generation and Telegram communication.
- Development time was limited to the project period.

3. Project Plan, Schedule and Progress Report

3.1 Team Roles and Responsibilities

Member	Role	Responsibilities
Kuldeep Sanodiya	Project coordinator	[Responsibilities]
Paras Mehra	AI / backend developer	[Responsibilities]
Anil Kumar	Telegram bot and database developer	[Responsibilities]
Gaurav Yadav	Tester / documentation	[Responsibilities]

3.2 Work Breakdown and Schedule

Phase / task	Planned period	Actual period	Status / evidence

Phase / task	Planned period	Actual period	Status / evidence
Problem definition and research	[Dates]	[Dates]	[Evidence]
Requirements and architecture	[Dates]	[Dates]	[Evidence]
Gemini integration and content generation	[Dates]	[Dates]	[Evidence]
Telegram bot, FastAPI backend and database	[Dates]	[Dates]	[Evidence]
Scheduling and festival content	[Dates]	[Dates]	[Evidence]
Testing and improvement	[Dates]	[Dates]	[Evidence]
Deployment and report	[Dates]	[Dates]	[Evidence]

3.3 Progress Monitoring

Progress can be reviewed weekly using a task list, version-control history and feedback from the project guide. [Describe how the team actually tracked progress, what was completed each week, what blocked the work, and any change from the original plan.]

3.4 Budget and Resources

Item	Estimated cost	Actual cost	Notes
Gemini API usage	[₹]	[₹]	[Free tier / usage-based]
Telegram Bot API	[₹]	[₹]	[Free / details]
Hosting or deployment	[₹]	[₹]	[Local machine / platform]
Other resources	[₹]	[₹]	[Details]
Total	[₹]	[₹]	[Actual total]

If no money was spent, state that clearly and list the free tools and existing equipment that were used.

4. AI Design and Architecture

4.1 Design Approach

Content Saathi is designed as an AI-assisted workflow with the owner in the loop. A request arrives through Telegram, the backend prepares the input from the shop details, Gemini generates the content, the result is saved as a draft, and the owner decides what happens next: regenerate, approve or reject. In addition to responding to commands, the system can also generate content on its own at configured times and can prepare festival posts, which makes it more than a one-time question-and-answer chatbot. Final decisions always remain with the owner.

4.2 System Architecture

The system follows the flow:

User → Telegram Bot → Python / FastAPI → Gemini AI → SQLite DB → Response.

Layer / component	Responsibility
User	Sends a request or query through Telegram
Telegram Bot	Receives the command or query and returns the result to the user
Python backend	Processes the request and prepares the input for the AI model
FastAPI	Handles the relevant API operations
Gemini AI	Generates the promotional content or the response to a query
SQLite database	Stores generated posts and their status
JSON configuration	Holds shop information and settings
Scheduler	Triggers content generation at configured times

[Insert the architecture diagram here as Figure 1.]

4.3 Working Process

1. The user sends a command or query through Telegram.
2. The application receives and processes the request, combining it with the saved shop information.
3. Gemini AI generates a response or promotional post.
4. The generated post is saved in the database with the status Draft.
5. The user reviews the draft and regenerates it if required.
6. The user approves or rejects the draft, and the status is updated in the database.

4.4 Database Design

Generated posts are stored in a local SQLite database (database/content_saathi.db). Each record keeps the generated post and its current status.

Status	Meaning
DRAFT	Post has been generated and is waiting for review
APPROVED	The owner has approved the content
REJECTED	The owner has rejected the content

[List the actual table name and columns used, for example post id, content, platform format, status and creation time.]

4.5 Key Components

Component	Responsibility	Implementation to document
Request input	Receive commands and queries	Telegram bot commands / messages
Content generator	Build the prompt and obtain the post from Gemini	[Prompt design and model name]
Format handling	Produce Telegram, Instagram and Google Business versions	[How formats are selected]
Draft manager	Save, regenerate, approve and reject posts	[Functions and database operations]
Scheduler	Generate content at configured times	[Schedule configuration]
Festival module	Create festival posts using verified dates	[Source of verified dates]
Configuration	Shop details and settings	JSON files and environment variables

4.6 Safety and Human Oversight

- The owner reviews every generated post and decides whether to approve or reject it.
- API keys and the Telegram bot token are kept in environment variables and never written in the source code or this report.
- Festival posts use verified dates to avoid announcing wrong festival information.
- Errors from the Gemini API or Telegram are reported rather than hidden.
- Only the shop information necessary for generating content is stored.
- Generated text may contain mistakes, so the owner is expected to check it before use.

4.7 Data Flow and Storage

The application stores two kinds of data: shop information and configuration in JSON files, and generated posts with their status in the SQLite database on the local machine. Session data is handled by the Telegram bot. [State exactly what is saved, where, who can access it and how it can be removed.]

5. Implementation

5.1 Technology Stack

Technology	Purpose
Python	Main programming language
Google Gemini AI	Content generation and AI responses
FastAPI	Backend API development
SQLite	Storing generated posts
Telegram Bot API	User communication
JSON	Storing shop information and configuration
Python Scheduler	Scheduling content-generation tasks
Uvicorn	Running the FastAPI server
Python-dotenv	Managing environment variables

5.2 Key Features and Implemented Modules

Module	Description	Status	Evidence
AI Content Generation	Creates posts from shop information and user instructions	[Complete / Partial / Planned]	[Screenshot / sample output]
Telegram Bot Integration	Lets users interact with the application through Telegram	[Status]	[Screenshot]
AI Query Responses	The configured Gemini model answers questions	[Status]	[Screenshot]
Regenerate Content	Produces a new version when a draft is not satisfactory	[Status]	[Screenshot / test ID]
Draft Management	Review, approval and rejection of generated posts	[Status]	[Screenshot / test ID]
Database Storage	Saves posts and statuses in SQLite	[Status]	[Database screenshot]
Scheduled Generation	Automatic generation at configured times	[Status]	[Log / test ID]
Festival-Based Content	Festival posts using verified dates	[Status]	[Sample output]

The eight modules above are the features described in the project presentation. Mark each one honestly as Complete, Partial or Planned and attach supporting evidence.

5.3 Prompt Design

The prompt sent to Gemini is built from the shop information stored in JSON, the type of post requested (for example festival greeting or offer), the target format (Telegram, Instagram or Google Business) and the owner's instructions. [Summarise the actual instructions given to the model, including role, tone, output format and restrictions. Include a sanitised example. Do not include API keys.]

5.4 User Interface and Demonstration

The interface of Content Saathi is the Telegram chat itself. [Insert labelled screenshots as figures: starting the bot, requesting a post, the generated draft, regeneration, approval and rejection, an AI query response, and a scheduled or festival post. Give each figure a number and a short caption.]

5.5 Project File Structure

[Insert a folder tree of the actual repository and explain the main files, such as the FastAPI application, the Telegram bot handler, the Gemini content module, the scheduler, the JSON configuration, the .env file (placeholders only) and database/content_saathi.db.]

6. Testing, Test Cases and Results

6.1 Testing Approach

The complete workflow, from Telegram request to saved and approved post, is tested along with individual components. Tests cover normal requests, unclear or incomplete requests, regeneration, approval and rejection, scheduled generation, festival posts and failure conditions such as an unavailable API. The expected result is written before each test is run and the observed result is then recorded honestly.

6.2 Test Cases

ID	Scenario	Expected result	Actual result	Status
TC-01	Request a promotional post through Telegram	A post is generated using the shop details and request and returned in the chat	[Record actual result]	Pass
TC-02	Request a post in Instagram format	The post is returned in the Instagram format	[Record actual result]	Pass
TC-03	Request a post in Google Business format	The post is returned in the Google Business format	[Record actual result]	Pass
TC-04	Ask a general question to the bot	Gemini returns a relevant answer	[Record actual result]	Pass
TC-05	Regenerate an unsatisfactory draft	A new version is generated and the draft record is updated or added	[Record actual result]	Pass
TC-06	Approve a draft	The status changes to APPROVED in SQLite	[Record actual result]	Pass
TC-07	Reject a draft	The status changes to REJECTED in SQLite	[Record actual result]	Pass
TC-08	Scheduled generation at a configured time	A post is generated automatically at that time and saved as DRAFT	[Record actual result]	Pass
TC-09	Festival post request	The post uses the verified festival date	[Record actual result]	Pass
TC-10	Gemini API unavailable or key invalid	A clear error message is shown and no post is falsely saved as successful	[Record actual result]	Pass

6.3 Results Summary

Metric	How to calculate / evaluate	Actual result
Task completion	Successfully completed requests ÷ requests tested	[Value / not measured]
Response time	Record method, number of runs and average or median	[Value / not measured]
Content quality	Reviewer checklist (relevance, tone, correct shop details, correct format)	[Value / not measured]
Status handling	Approvals and rejections correctly stored ÷ attempted	[Value / not measured]
Failure handling	Failure scenarios handled correctly ÷ scenarios tested	[Value / not measured]

The rows above are evaluation methods, not pre-filled results. Claim accuracy, speed or user satisfaction only if evidence has been collected.

6.4 Requirement-to-Test Mapping

Requirement	Related test case	Outcome / evidence
FR-01	TC-01	[Actual outcome]
FR-02	TC-01, TC-04	[Actual outcome]
FR-03	TC-04	[Actual outcome]
FR-04	TC-02, TC-03	[Actual outcome]
FR-05	TC-05	[Actual outcome]
FR-06	TC-06, TC-07	[Actual outcome]
FR-07	TC-06, TC-07	[Actual outcome]

Requirement	Related test case	Outcome / evidence
FR-08	TC-08	[Actual outcome]
FR-09	TC-09	[Actual outcome]

7. Deployment

7.1 Deployment Environment

The application runs as a FastAPI server using Uvicorn, connected to a Telegram bot and the Gemini API, with a local SQLite database. [State the actual environment: operating system, Python version, whether it runs on a local machine, college server or cloud platform, and the access required. Do not include real secrets.]

7.2 Setup and Run Instructions

1. Install Python and the required packages (FastAPI, Uvicorn, python-dotenv, the Gemini client library, the Telegram bot library and the scheduler) as listed in the project's requirements file.
2. Create a Telegram bot and obtain its token; obtain a Gemini API key.
3. Store the token and key in a local .env file using placeholder names in this report, for example TELEGRAM_BOT_TOKEN and GEMINI_API_KEY. Never commit the real values.
4. Fill in the shop information and settings in the JSON configuration file.
5. Start the FastAPI server with Uvicorn and start the Telegram bot as documented in the project (the SQLite database is created at database/content_saathi.db).
6. Open Telegram, send a test request to the bot and confirm that a draft is returned and saved.

7.3 Deployment Evidence and Limitations

[Insert a screenshot or log of the running application with date and environment.] Known limitations include dependence on internet access, Gemini API quota and rate limits, a local SQLite database suited to a single shop, and the absence of production monitoring.

8. Problems Encountered and Solutions

Record the real issues faced by the team. The rows below are prompts only and must be replaced with problems that actually occurred.

Problem	Investigation / cause	Impact	Solution	Learning
[Example: AI output not matching the requested format]	[What testing showed]	[Actual impact]	[Prompt or format change]	[Prevention next time]
[Example: Festival date wrong or unclear]	[Cause]	[Actual impact]	[Use of verified dates]	[Verify before generating]
[Example: API key / quota or network error]	[Cause]	[Actual impact]	[Error handling or retry]	[Do not assume success]
[Example: Scheduler not triggering at the configured time]	[Cause]	[Actual impact]	[Fix]	[Learning]

8.1 Risk Management

Risk	Likelihood	Impact	Mitigation
Incorrect, off-brand or inappropriate AI-generated content	[Low/Med/High]	[Low/Med/High]	Owner reviews every draft; regenerate or reject when unsuitable

Risk	Likelihood	Impact	Mitigation
Gemini API or Telegram unavailable, or quota exhausted	[Low/Med/High]	[Low/Med/High]	Clear error handling; retry later
Exposure of API key or bot token	[Low/Med/High]	[Low/Med/High]	Environment variables; keep .env out of version control
Wrong festival information	[Low/Med/High]	[Low/Med/High]	Use verified festival dates
Loss of the local database	[Low/Med/High]	[Low/Med/High]	Take periodic backups of the SQLite file

9. Results, Limitations and Future Scope

9.1 Results Against Objectives

Objective	Target	Actual result	Status / evidence
Generate promotional content using AI	[Target]	[Actual result]	[Achieved / partial / not achieved]
Accept requests and queries through Telegram	[Target]	[Actual result]	[Evidence]
Support Telegram, Instagram and Google Business formats	[Target]	[Actual result]	[Evidence]
Regenerate content when needed	[Target]	[Actual result]	[Evidence]
Store posts in SQLite	[Target]	[Actual result]	[Evidence]
Provide draft approval and rejection	[Target]	[Actual result]	[Evidence]
Support scheduled content generation	[Target]	[Actual result]	[Evidence]

9.2 Benefits

- Saves time and effort in content creation.
- Makes AI accessible to small businesses.
- Easy interaction through Telegram.
- Helps maintain regular promotion.
- Stores posts for convenient tracking.
- Keeps the owner involved in approval.

These are the expected benefits of the system. Time saved and user satisfaction should be described as proven only if they were measured.

9.3 Limitations

- Generated content may need editing, and its quality depends on the model and the quality of the shop details and instructions.
- The system depends on internet access, API availability and quota limits.
- The local SQLite database is intended for a single shop and a small amount of data.
- Posts are not published automatically to social platforms; the owner uses the approved content.
- The system cannot guarantee increased sales or customer engagement.

9.4 Future Enhancements

Enhancement	Expected benefit	Priority
Web dashboard for content management	Easier viewing and editing of all posts	[Priority]
More language and tone options	Content suited to more customers and brands	[Priority]
Improved festival calendar verification	More reliable festival posts	[Priority]

Enhancement	Expected benefit	Priority
Generation and approval analytics	Insight into which content is accepted or rejected	[Priority]
Authorized social media publishing	Publish approved posts without copy-pasting	[Priority]
Multiple shops and business accounts	Use by more businesses from one system	[Priority]

10. Conclusion and Learning

Content Saathi is an AI-powered application that simplifies promotional content creation for small businesses. It combines Python, Google Gemini AI, FastAPI, SQLite and the Telegram Bot API to generate content, answer queries, store drafts and support the owner's review through approval, rejection and regeneration. By keeping the owner in control and making content creation possible from a simple chat, the project aims to make regular promotion easier and more accessible for small shops. [Before submission, add a sentence on which features were completed, what the tests showed and what limitations remain.]

10.1 Learning Outcomes

- **Technical learning:** Learned Gemini API, prompt design, FastAPI, Telegram Bot API, SQLite, and Python scheduling.
- **Project management:** Improved teamwork, planning, task division, and progress tracking.
- **Responsible AI:** Learned to review AI-generated content, protect API keys, and keep humans in control of publishing

10.2 Individual Contributions

Member	Contribution	Learning / responsibility
Kuldeep Sanodiya	[Specific modules or tasks completed]	[Learning]
Paras Mehra	[Specific modules or tasks completed]	[Learning]
Anil Kumar	[Specific modules or tasks completed]	[Learning]
Gaurav Yadav	[Specific modules or tasks completed]	[Learning]

11. References

No.	Author / organization	Title	URL / access date
1	Google	Gemini API documentation	https://ai.google.dev/gemini-api/docs – accessed [date]
2	FastAPI project	FastAPI documentation	https://fastapi.tiangolo.com – accessed [date]
3	Telegram	Telegram Bot API	https://core.telegram.org/bots/api – accessed [date]
4	SQLite Consortium	SQLite documentation	https://www.sqlite.org/docs.html – accessed [date]
5	Encode	Uvicorn documentation	https://www.uvicorn.org – accessed [date]

12. Appendices

Appendix A: Sample Interaction

Replace this illustration with a real interaction from the working system.

Step	Interaction
User request	The owner sends a request on Telegram for a promotional post in Instagram format.
Processing	The backend combines the request with the saved shop details and sends it to Gemini.

Step	Interaction
Draft	The generated post is saved as DRAFT and shown to the owner.
Regeneration	The owner is not fully satisfied and asks for a new version; a fresh draft is returned.
Decision	The owner approves the post; the status in SQLite changes to APPROVED.

Appendix B: Screenshots and Code Excerpts





